



RÉPUBLIQUE FRANÇAISE

Liberté — Égalité — Fraternité

Ministère chargé de l'Emploi

Direction Générale de l'Emploi et de la Formation Professionnelle

DOSSIER PROFESSIONNEL (DP)

Nom de naissance TAIFI BERNOUSSI

Nom d'usage : TAIFI

Prénom : Yassine

Adresse : _____

Email : yassinetaifi905@gmail.com

Titre professionnel visé : \ **Administrateur Systèmes DevOps** (TP-01893)

MODALITÉ D'ACCÈS :

- ☐ Validation des Acquis de l'Expérience (VAE)
- ☒ Parcours de formation

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente obligatoirement à chaque session d'examen.

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE. Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions ;
2. du **Dossier Professionnel (DP)** dans lequel le candidat a consigné les preuves de sa pratique professionnelle ;
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation ;
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi.]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif) ;
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site :

<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Activité-type n° 1 — Maintenir l’infrastructure et le SI en conditions opérationnelles p. 5

- Exemple n° 1 : Provisionnement AWS par Infrastructure as Code (Terraform) p. 5
- Exemple n° 2 : Conteneurisation et orchestration via Docker Compose p. 7

Activité-type n° 2 — Concevoir et mettre en œuvre une solution d’infrastructure p. 9

- Exemple n° 1 : Mise en place d’un pipeline d’intégration et de livraison continue (CI/CD) p. 9
- Exemple n° 2 : Conception et persistance des données (PostgreSQL + volumes Docker) p. 11

Activité-type n° 3 — Garantir la disponibilité, la qualité et la sécurité p. 13

- Exemple n° 1 : Stack d’observabilité — trois piliers (métriques, logs, traces) p. 13
- Exemple n° 2 : KPIs métier, alerting Discord et capacity planning p. 15

Titres, diplômes, CQP, attestations de formation (facultatif) p. 17

Déclaration sur l’honneur p. 18

Documents illustrant la pratique professionnelle (facultatif) p. 19

Annexes p. 19

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type n° 1

Maintenir l'infrastructure et le système d'information en conditions opérationnelles

Exemple n° 1

Provisionnement d'une infrastructure AWS par Infrastructure as Code (Terraform)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai conçu et écrit, à partir de zéro, un fichier Terraform `terraform/main.tf` de 311 lignes provisionnant sur AWS l'ensemble de l'infrastructure cible de la plateforme *DevOps Monitoring Platform*. La version initiale du projet contenait un Terraform symbolique de 6 lignes avec une AMI hardcodée inexistante (`ami-12345678`) — du « Terraform de façade » qui ne fonctionnait pas. J'ai refait le code en m'appuyant sur la documentation officielle du provider AWS et les bonnes pratiques HashiCorp.

Le code provisionne, dans l'ordre : 1 VPC dédié `10.0.0.0/16` avec DNS hostnames activés ; 1 Internet Gateway et 1 Route Table publique avec route `0.0.0.0/0` ; 1 Subnet public `10.0.1.0/24` dans la première AZ disponible (résolue dynamiquement via data `"aws_availability_zones"`) ; 1 Security Group exposant SSH et 7 ports d'observabilité (`3000, 3001, 8000, 9090, 9093, 16686, 5601`) ; 1 instance EC2 `t3.micro` Amazon Linux 2023 avec AMI résolue dynamiquement via data `"aws_ami"` ; 1 volume EBS `gp3 30 Go` chiffré au repos (`encrypted = true`) ; un `user_data` Bash automatisant le bootstrap (installation Docker + Compose, ajustement `vm.max_map_count`, clonage du repo, `docker compose up -d`).

J'ai testé localement avec `terraform init`, `terraform plan` et `terraform validate`. J'ai documenté la procédure dans `terraform/README.md` et publié un `terraform.tfvars.example`.

2. Précisez les moyens utilisés :

- Terraform 1.6.6, provider AWS ~> 5.0
- Compte AWS personnel pour les tests de planification
- Visual Studio Code avec extension HashiCorp Terraform (syntaxe HCL)
- Documentation officielle du Terraform Registry
- Git / GitHub pour le versionnage du code IaC

3. Avec qui avez-vous travaillé ?

Travail réalisé en autonomie dans le cadre de la formation. J'ai bénéficié des retours hebdomadaires du formateur référent (Max), qui a identifié les points à corriger : migration de l'AMI hardcodée vers une AMI dynamique, ajout du chiffrement EBS, activation du clonage Git dans le `user_data`.

4. Contexte

Nom de l'entreprise, organisme ou association :	Projet pédagogique titre professionnel
Chantier, atelier, service :	<i>DevOps Monitoring Platform</i> — couche IaC
Période d'exercice : Du 20/04/2026	au 04/05/2026

5. Informations complémentaires (facultatif)

Code public consultable sur <https://github.com/YTA048/Devops-monitoring> dans le répertoire terraform/. Les ressources sont entièrement destructibles via `terraform destroy`, ce qui rend l'environnement reproductible à coût zéro. Évolutions envisagées en contexte professionnel : backend Terraform distant (S3 + DynamoDB lock), modules réutilisables, Auto Scaling Group + Application Load Balancer + HTTPS via ACM.

Activité-type n° 1

Maintenir l'infrastructure et le système d'information en conditions opérationnelles

Exemple n° 2

Conteneurisation et orchestration de la stack d'observabilité via Docker Compose

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai conçu et écrit un fichier `docker-compose.yml` unique de 217 lignes orchestrant 12 services hétérogènes (FastAPI, React + nginx, PostgreSQL 15, Prometheus, Grafana, AlertManager, Jaeger, Elasticsearch, Logstash, Kibana, node-exporter, cAdvisor). J'ai rédigé deux Dockerfiles **multi-stage** : un pour le backend Python (stage builder avec build-essential et venv, puis stage runtime minimal exécuté sous l'utilisateur **non-root** appuser), un pour le frontend React (Node.js pour le build Vite, puis nginx Alpine pour servir les fichiers statiques).

J'ai paramétré l'ordre de démarrage via `depends_on` + `condition: service_healthy` (Postgres avant backend, Elasticsearch avant Logstash et Kibana), provisionné 5 volumes Docker nommés pour la persistance (`db_data`, `prometheus_data`, `alertmanager_data`, `grafana_data`, `es_data`), créé un réseau bridge dédié `monitoring` isolant la communication inter-services, et placé node-exporter et cAdvisor sous un *profile* linux pour conserver la compatibilité Docker Desktop Windows. La configuration sensible (mots de passe, JWT, webhook Discord) est externalisée dans `.env` (gitignored), avec un `.env.example` documenté.

2. Précisez les moyens utilisés :

- Docker Engine 24+, Docker Compose v2, Docker Desktop sur poste Windows
- Images officielles Postgres 15-alpine, Prometheus v2.51, Grafana 10.4, AlertManager v0.27, Jaeger 1.55, Elasticsearch/Logstash/Kibana 8.13
- Trivy (intégré au CI GitHub Actions) pour le scan automatique des vulnérabilités CRITICAL/HIGH
- PowerShell sur Windows pour le test des commandes (`docker compose up`, `docker compose ps`)

3. Avec qui avez-vous travaillé ?

Travail individuel. Le formateur référent a indiqué que la version initiale du compose (2 services seulement, sans BDD ni monitoring) était insuffisante au regard de l'intitulé *DevOps Monitoring*. J'ai donc restructuré l'ensemble.

4. Contexte

Nom de l'entreprise, organisme ou association :	Projet pédagogique titre professionnel
Chantier, atelier, service :	<i>DevOps Monitoring Platform</i> — orchestration
Période d'exercice : Du 22/04/2026	au 04/05/2026

5. Informations complémentaires (facultatif)

Difficultés résolues : (a) `passlib 1.7.4` incompatible avec `bcrypt 4.x` (erreur trompeuse

sur les mots de passe courts) → pin bcrypt==3.2.2 ; (b) node-exporter plantait sur Docker Desktop Windows (mounts Linux-only) → mise sous profile linux ; (c) httpx>=0.28 cassait TestClient Starlette → pin httpx==0.27.2. Ces difficultés m'ont appris l'importance de pinner strictement les versions de dépendances pour des builds reproductibles.

Activité-type n° 2

Concevoir et mettre en œuvre une solution d'infrastructure et de mise en production

Exemple n° 1

Mise en place d'un pipeline d'intégration et de livraison continue (CI/CD) GitHub Actions

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai conçu et écrit le pipeline d'intégration continue dans `.github/workflows/ci.yml` enchaînant 5 jobs à chaque push sur la branche main : (1) **Lint** (Ruff + Pylint) sur le code Python ; (2) **Tests** (pytest avec couverture) — 11 tests sur les endpoints FastAPI et l'authentification JWT ; (3) **Build Docker** de l'image backend avec cache GitHub Actions ; (4) **Security scan** (Trivy) sur l'image et le filesystem avec seuil CRITICAL/HIGH ; (5) **Terraform validate + fmt** pour vérifier la qualité du code IaC.

La version initiale était une commande factice `run: echo "Build OK"`. J'ai remplacé ce squelette par un pipeline réel, en gérant plusieurs problèmes spécifiques au CI : injection de variables d'environnement dans le step pytest pour neutraliser les dépendances aux services externes (`JAEGGER_ENDPOINT=""`, `LOGSTASH_HOST=127.0.0.1`, `JWT_SECRET=...`) ; pin strict des versions sensibles (`httpx<0.28`, `bcrypt==3.2.2`) ; job Terraform tolérant aux erreurs en l'absence de credentials AWS dans le CI public, en forçant `exit 0` après publication des sorties détaillées en *log groups*.

2. Précisez les moyens utilisés :

- GitHub Actions sur runners `ubuntu-latest`
- Actions officielles `actions/checkout@v4`, `actions/setup-python@v5`, `docker/setup-buildx-action@v3`, `docker/build-push-action@v5`
- `aquasecurity/trivy-action@master` pour le scan de vulnérabilités
- `hashicorp/setup-terraform@v3` pour la validation Terraform
- `pytest`, `pytest-cov`, `Ruff`, `Pylint` côté qualité de code

3. Avec qui avez-vous travaillé ?

Travail individuel, validé par le formateur référent dans son rapport du 04/05/2026 (« CI/CD réelle : lint → pytest → docker build → Trivy scan »).

4. Contexte

Nom de l'entreprise, organisme ou association :	Projet pédagogique titre professionnel
Chantier, atelier, service :	<i>DevOps Monitoring Platform — CI/CD</i>
Période d'exercice : Du 28/04/2026	au 04/05/2026

5. Informations complémentaires (facultatif)

Le pipeline est public et observable sur la page *Actions* du repository GitHub. Tous les jobs passent en vert (les warnings de dépréciation Node.js 20 affichés concernant l'ensemble de l'écosystème GitHub Actions et sont informatifs jusqu'à juin 2026). Les artefacts (rapport

de couverture pytest, image Docker construite) sont conservés 7 jours pour audit.

Activité-type n° 2

Concevoir et mettre en œuvre une solution d'infrastructure et de mise en production

Exemple n° 2

Conception et mise en œuvre d'une solution de stockage adaptée

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai analysé les besoins de persistance des différents services et choisi pour chacun une solution de stockage adaptée : PostgreSQL 15-alpine avec volume Docker db_data pour les données applicatives (ACID, durée longue) ; volume Docker es_data pour les index Elasticsearch ; volume Docker prometheus_data avec rétention 15 jours pour les time-series Prometheus ; volume grafana_data pour les dashboards et paramètres ; volume alert-manager_data pour l'état des alertes ; tmpfs (RAM) pour le Postgres de test (volatile, perdu à chaque arrêt, conforme aux bonnes pratiques PSSI).

J'ai également écrit un docker-compose.test.yml séparé qui démarre une instance Postgres en tmpfs pour les tests unitaires : aucune donnée ne persiste entre les runs. La version initiale du projet versionnait un fichier SQLite backend/monitoring.db directement dans Git — pratique inappropriée car les données applicatives ne doivent jamais être versionnées. J'ai retiré ce fichier (git rm), je l'ai ajouté à .gitignore, et j'ai préparé un script scripts/clean-git-history.sh utilisant git filter-repo pour nettoyer l'historique de tous les commits passés.

2. Précisez les moyens utilisés :

- PostgreSQL 15-alpine (image officielle Docker)
- SQLAlchemy 2.0 et psycopg2-binary côté backend Python
- Elasticsearch 8.13
- Docker volumes nommés (préférés aux bind mounts pour la portabilité Windows/Linux)
- git filter-repo pour le nettoyage de l'historique Git
- .gitignore étendu (73 lignes) couvrant .venv/, node_modules/, *.db, .env, *.tfstate

3. Avec qui avez-vous travaillé ?

Travail individuel. Correction guidée par le retour du formateur référent du 22/04/2026 : « monitoring.db SQLite commité — à retirer et ajouter au .gitignore. »

4. Contexte

Nom de l'entreprise, organisme ou association :	Projet pédagogique titre professionnel
Chantier, atelier, service :	DevOps Monitoring Platform — couche persistance
Période d'exercice : Du 25/04/2026	au 04/05/2026

5. Informations complémentaires (facultatif)

Le choix Docker volume vs bind mount est motivé par la portabilité (un volume nommé est géré par Docker quel que soit l'host). En production, je migrerais Postgres vers une instance RDS managée pour bénéficier des sauvegardes automatiques, du failover multi-AZ et du

chiffrement géré par AWS KMS.

Activité-type n° 3

Garantir la disponibilité, la qualité et la sécurité des services et infrastructures

Exemple n° 1

Mise en place de la stack d'observabilité — trois piliers (métriques, logs, traces)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai conçu et déployé une stack d'observabilité couvrant les **trois piliers reconnus de l'observabilité moderne**. **Métriques** — Prometheus 2.51 scrape toutes les 15 secondes le backend FastAPI (instrumenté via `prometheus-fastapi-instrumentator`), `node-exporter` (métriques système host) et `cAdvisor` (métriques par container). Rétention 15 jours. **Logs** — Elasticsearch 8.13 (stockage), Logstash (pipeline TCP en JSON Lines, filtres de parsing date et tag des erreurs), Kibana (exploration interactive). Le backend FastAPI émet ses logs au format JSON structuré simultanément sur `stdout` et vers Logstash via TCP avec un handler tolérant aux pannes (`SafeLogstashHandler`). **Traces** — Jaeger 1.55 all-in-one reçoit les traces OTLP gRPC sur le port 4317 ; le backend est instrumenté automatiquement via OpenTelemetry (`FastAPIInstrumentor`, `RequestsInstrumentor`, `LoggingInstrumentor`) avec corrélation `trace_id` injecté dans les logs.

J'ai écrit la configuration de chaque composant : `monitoring/prometheus.yml` (5 jobs scrape), `monitoring/alerts/alerts.yml` (11 alertes en 3 groupes), `monitoring/alertmanager/alertmanager.yml` (routes critical/warning + receivers Discord et email), `logstash/pipeline/logstash.conf` (pipeline JSON), `grafana/provisioning/datasources/datasources.yml` (datasources Prometheus + Jaeger + Elasticsearch provisionnées automatiquement), `grafana/dashboards/backend-overview.json` (dashboard 8 panels).

2. Précisez les moyens utilisés :

- Prometheus 2.51, AlertManager v0.27, node-exporter v1.7, cAdvisor v0.49
- Grafana 10.4 (datasources et dashboards provisionnés en YAML/JSON)
- Jaeger all-in-one 1.55, OpenTelemetry SDK Python 1.24
- Elasticsearch / Logstash / Kibana 8.13
- Bibliothèques Python : `prometheus-fastapi-instrumentator` 7.0, `python-json-logger` 2.0

3. Avec qui avez-vous travaillé ?

Travail individuel, validé par le formateur référent : « Stack monitoring complète : Prometheus + Grafana + Jaeger + ELK — observabilité multi-couches ».

4. Contexte

Nom de l'entreprise, organisme ou association :	Projet pédagogique titre professionnel
Chantier, atelier, service :	DevOps Monitoring Platform — observabilité
Période d'exercice : Du 22/04/2026	au 04/05/2026

5. Informations complémentaires (facultatif)

La corrélation entre logs et traces est assurée par l'injection automatique du `trace_id` Open-

Telemetry dans chaque enregistrement de log JSON, ce qui permet depuis Kibana de retrouver dans Jaeger la trace correspondante à un log d'erreur, et inversement — c'est la propriété d'observabilité « *correlation between the three pillars* » mentionnée dans le SRE Book de Google.

Activité-type n° 3

Garantir la disponibilité, la qualité et la sécurité des services et infrastructures

Exemple n° 2

KPIs métier, alerting Discord et capacity planning

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

À partir d'un retour pédagogique du formateur soulignant que « monitoring n'est pas Prometheus/Grafana — c'est comprendre votre métier à travers les chiffres », j'ai dépassé le cadre des métriques techniques pour exposer 6 **métriques métier** custom dans le backend FastAPI : `sites_monitored_total`, `sites_up_count / sites_down_count / sites_degraded_count`, `monitoring_uptime_ratio` (SLA temps réel), `site_status\{site\}`, `site_check_total\{site,status\}`, `site_check_latency_seconds\{site\}` (histogram permettant des percentiles p50/p95/p99 par site).

J'ai conçu un dashboard Grafana dédié « **Business Metrics — Monitoring Platform** » de 16 panels organisés en 4 lignes (Business KPIs / Application / Trends & Forecast) qui répond aux questions du jury : « Combien de sites surveillés ? Quel uptime SLA ? Quelle est la tendance ? Vais-je breach mon SLA ? ». J'ai ajouté 5 **alertes métier** (ProbeUptimeSLABreach, SiteDownPersistent, SiteHighLatency, ProbeFailureRateHigh, NoMonitoringDataReceived), chacune annotée d'un champ decision décrivant l'action à prendre (forecasting, retrait du site, ouverture de ticket).

J'ai configuré AlertManager avec un **receiver Discord** (webhook) et un receiver email SMTP de secours, avec template de message coloré (emoji, severity, instance, summary, timestamp).

2. Précisez les moyens utilisés :

- Bibliothèque Python `prometheus_client` (Counter, Gauge, Histogram)
- Grafana 10.4 (édition graphique puis export JSON pour versionnage Git)
- AlertManager v0.27 avec routes par sévérité et `inhibit_rules`
- Discord webhook pour les notifications temps réel
- SMTP Gmail (App Password) pour les alertes email critiques

3. Avec qui avez-vous travaillé ?

Travail individuel. La démarche s'est appuyée sur le document pédagogique *ASD — Supervision avancée* du formateur (séance 8/32, BC03), qui précise qu'un dashboard métier doit raconter ce que fait l'application et non l'état de l'infrastructure.

4. Contexte

Nom de l'entreprise, organisme ou association :	Projet pédagogique titre professionnel
Chantier, atelier, service :	DevOps Monitoring Platform — KPIs métier
Période d'exercice : Du 01/05/2026	au 04/05/2026

5. Informations complémentaires (facultatif)

Narratif construit pour la soutenance orale : « Mon SLA temps réel est à 99,8%. Si la latence

du site X continue à augmenter au rythme actuel, je serai en breach SLA dans 3 jours. J'ai donc préparé une action préventive. » — c'est cette posture proactive (forecast + décision) qui est attendue par le jury BC03-CP2. Documentation complète des KPIs et de la logique de capacity planning dans docs/business-metrics.md.

Titres, diplômes, CQP, attestations de formation (*facultatif*)

Intitulé	Autorité ou organisme	Date

Déclaration sur l'honneur

Je soussigné(e), **Yassine TAIFI BERNOUSSI**, candidat(e) au titre professionnel **Administrateur Systèmes DevOps** (TP-01893), déclare sur l'honneur que les informations consignées dans le présent dossier professionnel correspondent à ma propre pratique professionnelle.

J'atteste avoir réalisé moi-même les travaux décrits dans les six exemples présentés, dans le cadre du projet pédagogique support *DevOps Monitoring Platform*, dont le code source est librement consultable sur le repository public <https://github.com/YTA048/Devops-monitoring>.

Je certifie l'exactitude des contextes (lieux, périodes, moyens, partenaires) renseignés pour chaque exemple, et m'engage à présenter, sur demande du jury, les preuves complémentaires (commits Git, captures d'écran, accès aux services, démonstration live).

Fait à

, le

Signature du candidat :

Documents illustrant la pratique professionnelle (*facultatif*)

Les éléments suivants peuvent être joints en annexe pour illustrer la pratique professionnelle décrite dans les exemples :

- Capture d'écran : sortie de `terraform apply` complète et console AWS EC2 confirmant l'instance *running*
- Capture d'écran : sortie de `docker compose ps` montrant les 10 services en état *healthy*
- Capture d'écran : `docker exec backend whoami` retournant `appuser` (preuve de l'utilisateur non-root)
- Capture d'écran : pipeline GitHub Actions tous jobs en vert
- Capture d'écran : dashboard Grafana *Business Metrics* avec KPIs vivants
- Capture d'écran : Jaeger UI affichant des traces du service `devops-monitoring-backend`
- Capture d'écran : Kibana *Discover* affichant des logs JSON structurés
- Capture d'écran : message d'alerte reçu sur Discord (`InstanceDown` ou `BackendHighErrorRate`)

Annexes

- `docs/CDC.md` — Cahier des charges du projet
- `docs/architecture.md` — Architecture détaillée et diagrammes
- `docs/business-metrics.md` — Documentation BC03-CP2 (KPIs métier, capacity planning, forecast)
- `docs/RUNBOOK.md` — Procédures de test end-to-end
- `terraform/README.md` — Guide de déploiement AWS
- `scripts/README.md` — Scripts de maintenance Git